



Celebal Summer Internship 2026

Week 2 Task: E-Commerce Sales Database

Submitted By:- Saksham Sharma

1. Scenario & Business Context

I am a Junior Data Analyst at ShopEase, a mid-sized e-commerce company that sells electronics, clothing, and home products across India. The management team wants to understand sales patterns, customer behavior, and product performance to make better business decisions.

I had been given access to the company's relational database containing information about customers, products, orders, and order details. My task is to write SQL queries to extract meaningful insights from this data.

⚠ Note: I have used MySQL RDBMS to complete these tasks. All queries follow standard SQL syntax.

2. Database Schema & Constraints

The database consists of 4 tables.

Table: customers

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  city VARCHAR(50) NOT NULL,  
  state VARCHAR(50) NOT NULL,  
  join_date DATE NOT NULL,  
  is_premium BOOLEAN DEFAULT FALSE  
);  
  
-- Index for filtering by city/state  
CREATE INDEX idx_customers_city ON customers(city);  
CREATE INDEX idx_customers_state ON customers(state);
```

Table: products

```
CREATE TABLE products (  
  product_id INT PRIMARY KEY,  
  product_name VARCHAR(100) NOT NULL,  
  category VARCHAR(50) NOT NULL,  
  brand VARCHAR(50) NOT NULL,  
  unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),  
  stock_qty INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0)  
);  
  
-- Index for filtering by category  
CREATE INDEX idx_products_category ON products(category);
```

Table: orders

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  order_date DATE NOT NULL,  
  status VARCHAR(20) NOT NULL DEFAULT 'Pending'  
  CHECK (status IN ('Pending', 'Shipped', 'Delivered', 'Cancelled'))
```

```

    total_amount DECIMAL(12,2) NOT NULL CHECK (total_amount >= 0),

    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

-- Index for date-based filtering and sorting
CREATE INDEX idx_orders_date ON orders(order_date);
CREATE INDEX idx_orders_status ON orders(status);

```

Table: order_items

```

CREATE TABLE order_items (
    item_id      INT          PRIMARY KEY,
    order_id     INT          NOT NULL,
    product_id   INT          NOT NULL,
    quantity     INT          NOT NULL CHECK (quantity > 0),
    unit_price    DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),
    discount_pct  DECIMAL(5,2) DEFAULT 0 CHECK (discount_pct BETWEEN 0 AND 100),

    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

```

Entity Relationships

```

customers  —(1:N)—> orders
orders     —(1:N)—> order_items
products   —(1:N)—> order_items

customers.customer_id  ◀—FK— orders.customer_id
orders.order_id        ◀—FK— order_items.order_id
products.product_id    ◀—FK— order_items.product_id

```

3. Sample Data

Below is sample data to help understand the tables. Using the provided INSERT statements to load data into my database.

customers (sample rows)

customer_id	first_name	last_name	email	city	state	join_date	is_premium
1	rav	arma	rav.s@email.co	umbai	hharashtra	24-01-15	UE
2	ya	tel	ya.p@email.co	medabad	jarat	24-02-20	LSE
3	han	pta	han.g@email.c	lhi	lhi	24-03-10	UE
4	eha	ddy	eha.r@email.c	derabad	angana	24-04-05	LSE
5	ram	gh	ram.s@email.c	pur	asthan	24-05-12	UE
6	anya	r	anya.i@email.c	ennai	mil Nadu	24-06-18	LSE
7	ran	eha	ran.m@email.c	ne	hharashtra	24-07-22	UE
8	rya	ir	rya.n@email.co	chi	rala	24-08-30	LSE

products (sample rows)

product_id	product_name	category	brand	unit_price	stock_qty
1	Wireless Earbuds	Electronics	Apple	99.0	10
2	Cotton T-Shirt	Clothing	Nike	19.0	50
3	Smart Watch	Electronics	Apple	99.0	10
4	Running Shoes	Clothing	Nike	99.0	10
5	Bluetooth Speaker	Electronics	AmazonBasics	99.0	10
6	Bedsheet Set	Home	AmazonBasics	99.0	10
7	Tabletop Stand	Electronics	AmazonBasics	9.0	10
8	Phone Covers (Set)	Home	AmazonBasics	9.0	10

orders (sample rows)

order_id	customer_id	order_date	status	total_amount
01	1	24-08-01	Shipped	98.0
02	2	24-08-03	Shipped	9.0
03	3	24-08-05	Shipped	98.0
04	1	24-08-10	Shipped	99.0
05	4	24-08-12	Cancelled	99.0
06	5	24-08-15	Shipped	98.0
07	6	24-08-18	Pending	99.0
08	3	24-08-20	Shipped	9.0
09	7	24-08-25	Shipped	98.0
10	8	24-08-28	Shipped	98.0

order_items (sample rows)

item_id	order_id	product_id	quantity	unit_price	discount_pct
01	01	1	1	99.0	
02	01	7	1	9.0	
03	02	2	1	19.0	
04	03	3	1	99.0	
05	03	4	1	99.0	
06	04	5	1	99.0	
07	05	3	1	99.0	
08	06	1	1	99.0	
09	06	4	1	99.0	
10	07	6	1	99.0	
11	08	7	1	9.0	
12	09	5	1	99.0	
13	09	8	1	9.0	
14	10	6	1	99.0	
15	10	8	1	9.0	

Data Loading — INSERT Statements

Copying and executing the following SQL to load sample data into my database:

```
-- ===== INSERT: customers =====
```

```
INSERT INTO customers VALUES
```

```
(101, 'Aarav', 'Sharma', 'aarav.s@email.com', 'Mumbai', 'Maharashtra', '2024-01-15', TRUE),  
(102, 'Priya', 'Patel', 'priya.p@email.com', 'Ahmedabad', 'Gujarat', '2024-02-20', FALSE),  
(103, 'Rohan', 'Gupta', 'rohan.g@email.com', 'Delhi', 'Delhi', '2024-03-10', TRUE),  
(104, 'Sneha', 'Reddy', 'sneha.r@email.com', 'Hyderabad', 'Telangana', '2024-04-05', FALSE),  
(105, 'Vikram', 'Singh', 'vikram.s@email.com', 'Jaipur', 'Rajasthan', '2024-05-12', TRUE),  
(106, 'Ananya', 'Iyer', 'ananya.i@email.com', 'Chennai', 'Tamil Nadu', '2024-06-18', FALSE),
```

```
(107, 'Karan', 'Mehta', 'karan.m@email.com', 'Pune', 'Maharashtra', '2024-07-22', TRUE),
(108, 'Divya', 'Nair', 'divya.n@email.com', 'Kochi', 'Kerala', '2024-08-30', FALSE);
```

```
-- ===== INSERT: products =====
```

```
INSERT INTO products VALUES
```

```
(201, 'Wireless Earbuds', 'Electronics', 'BoAt', 1499.00, 250),
(202, 'Cotton T-Shirt', 'Clothing', 'Levis', 799.00, 500),
(203, 'Smart Watch', 'Electronics', 'Noise', 2999.00, 150),
(204, 'Running Shoes', 'Clothing', 'Nike', 4599.00, 120),
(205, 'Bluetooth Speaker', 'Electronics', 'JBL', 3499.00, 200),
(206, 'Bedsheet Set', 'Home', 'Spaces', 1299.00, 300),
(207, 'Laptop Stand', 'Electronics', 'AmazonBasics', 899.00, 180),
(208, 'Cushion Covers (Set)', 'Home', 'HomeCenter', 599.00, 400);
```

```
-- ===== INSERT: orders =====
```

```
INSERT INTO orders VALUES
```

```
(1001, 101, '2024-08-01', 'Delivered', 4498.00),
(1002, 102, '2024-08-03', 'Delivered', 799.00),
(1003, 103, '2024-08-05', 'Shipped', 7498.00),
(1004, 101, '2024-08-10', 'Delivered', 3499.00),
(1005, 104, '2024-08-12', 'Cancelled', 2999.00),
(1006, 105, '2024-08-15', 'Delivered', 5898.00),
(1007, 106, '2024-08-18', 'Pending', 1299.00),
(1008, 103, '2024-08-20', 'Delivered', 899.00),
(1009, 107, '2024-08-25', 'Shipped', 6098.00),
(1010, 108, '2024-08-28', 'Delivered', 1598.00);
```

```
-- ===== INSERT: order_items =====
```

```
INSERT INTO order_items VALUES
```

```
(5001, 1001, 201, 2, 1499.00, 0),
(5002, 1001, 207, 1, 899.00, 10),
(5003, 1002, 202, 1, 799.00, 0),
(5004, 1003, 203, 1, 2999.00, 0),
(5005, 1003, 204, 1, 4599.00, 5),
(5006, 1004, 205, 1, 3499.00, 0),
(5007, 1005, 203, 1, 2999.00, 0),
(5008, 1006, 201, 1, 1499.00, 10),
(5009, 1006, 204, 1, 4599.00, 5),
(5010, 1007, 206, 1, 1299.00, 0),
(5011, 1008, 207, 1, 899.00, 0),
(5012, 1009, 205, 1, 3499.00, 0),
(5013, 1009, 208, 2, 599.00, 15),
(5014, 1010, 206, 1, 1299.00, 0),
(5015, 1010, 208, 1, 599.00, 0);
```

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customer's table.

Ans.- `SELECT * FROM customers ;`

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q2. Retrieve only the first_name, last_name, and city of all customers.

Ans.- `SELECT first_name, last_name, city FROM customers;`

first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur
Ananya	Iyer	Chennai
Karan	Mehta	Pune
Divya	Nair	Kochi

Q3. List all unique categories available in the products table.

Ans.- `SELECT DISTINCT category FROM products;`

	category
▶	Clothing
	Electronics
	Home

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Ans.- Each table's Primary Key:

- customers → customer_id
- products → product_id
- orders → order_id
- order_items → item_id

A Primary Key must be:

Unique – No two rows can have the same primary key value. This ensures that every record can be identified individually.

NOT NULL – A primary key cannot contain NULL values because every record must have a valid identifier.

Example:

In the customers table, customer_id is the primary key. If two customers had the same customer_id, the database would not be able to distinguish between them. Similarly, if customer_id were NULL, the customer record could not be uniquely identified. Therefore, primary keys help maintain data integrity and ensure that each record in a table is uniquely identifiable.

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

Ans.- The email column in the customers table has two constraints:

- UNIQUE — no two customers can share the same email address.
- NOT NULL — every customer must provide an email; it cannot be left empty.

If you try to INSERT a duplicate email, the database raises a constraint-violation error:

ERROR: UNIQUE constraint failed: customers.email

The entire INSERT statement is rejected and no row is added.

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

Ans.- `INSERT INTO products VALUES (209, 'Test Product', 'Electronics', 'Brand', -50, 10);`

Error Code: 3819. Check constraint 'products_chk_1' is violated.

The CHECK (unit_price > 0) constraint on the products table rejects any price that is zero or negative. This protects data integrity by ensuring prices are always a valid positive number.

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

Ans.- `SELECT * FROM orders WHERE status = 'Delivered';`

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00
NULL	NULL	NULL	NULL	NULL

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

Ans.- `SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000;`

product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200
NULL	NULL	NULL	NULL	NULL	NULL

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

Ans.- `SELECT * FROM customers WHERE join_date >= '2024-01-01' AND join_date < '2025-01-01' AND state = 'Maharashtra';`

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

Ans.- `SELECT * FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25' AND status <> 'Cancelled';`

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
NULL	NULL	NULL	NULL	NULL

Q11. Explain what the index `idx_orders_date` does. How would it improve the performance of a query that filters orders by `order_date`? Write a sample query that would benefit from this index.

Ans.- `idx_orders_date` is a B-Tree index created on the `order_date` column of the `orders` table. Without it, every query filtering by date must perform a full table scan — reading every row one by one. With the index, the database engine can jump directly to the matching date range in $O(\log n)$ time, dramatically reducing I/O for large tables.

A query that benefits:

SELECT * FROM orders WHERE order_date = '2024-08-15';

order_id	customer_id	order_date	status	total_amount
1006	105	2024-08-15	Delivered	5898.00
NULL	NULL	NULL	NULL	NULL

With the index, the engine seeks to '2024-08-15' in the B-Tree and retrieves only matching rows. Without it, all 10+ rows (or millions in production) would be scanned.

Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024`; — would the index on `join_date` be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

Ans.- Original (non-SARGable) query: **SELECT * FROM customers WHERE YEAR(join_date) = 2024;**

Why the index is NOT used: The `YEAR()` function wraps the indexed column. The database engine cannot use the index because it does not store pre-computed `YEAR()` values, it must evaluate the function for every row, resulting in a full table scan.

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Index-friendly (SARGable) rewrite: **SELECT * FROM customers WHERE join_date >= '2024-01-01' AND join_date < '2025-01-01';**

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Now the column is bare on the left side of a range comparison. The engine can use the B-Tree index on `join_date` to directly scan only rows within that date range.

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

Ans.- `SELECT COUNT(*) AS total_orders FROM orders;`

total_orders
10

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

Ans.- `SELECT SUM(total_amount) AS total_revenue FROM orders WHERE status = 'Delivered';`

total_revenue
17191.00

Q15. Calculate the average unit_price of products in each category.

Ans.- `SELECT category, ROUND(AVG(unit_price), 2) AS avg_unit_price FROM products GROUP BY category;`

category	avg_unit_price
Clothing	2699.00
Electronics	2224.00
Home	949.00

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

Ans.- `SELECT status, COUNT(*) AS order_count, SUM(total_amount) AS total_revenue FROM orders GROUP BY status ORDER BY total_revenue DESC;`

status	order_count	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	1	1299.00

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

Ans.- `SELECT category, MAX(unit_price) AS most_expensive, MIN(unit_price) AS cheapest FROM products GROUP BY category;`

category	most_expensive	cheapest
Clothing	4599.00	799.00
Electronics	3499.00	899.00
Home	1299.00	599.00

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

Ans.- SELECT category, ROUND(AVG(unit_price), 2) AS avg_unit_price FROM products GROUP BY category HAVING AVG(unit_price) > 2000;

category	avg_unit_price
Clothing	2699.00
Electronics	2224.00

Section D — Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

Ans.- `SELECT o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount FROM orders o INNER JOIN customers c ON o.customer_id = c.customer_id;`

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

Ans.- `SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.total_amount FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;`

customer_id	first_name	last_name	order_id	order_date	total_amount
101	Aarav	Sharma	1001	2024-08-01	4498.00
101	Aarav	Sharma	1004	2024-08-10	3499.00
102	Priya	Patel	1002	2024-08-03	799.00
103	Rohan	Gupta	1003	2024-08-05	7498.00
103	Rohan	Gupta	1008	2024-08-20	899.00
104	Sneha	Reddy	1005	2024-08-12	2999.00
105	Vikram	Singh	1006	2024-08-15	5898.00
106	Ananya	Iyer	1007	2024-08-18	1299.00
107	Karan	Mehta	1009	2024-08-25	6098.00
108	Divya	Nair	1010	2024-08-28	1598.00

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

Ans.- `SELECT oi.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct FROM order_items oi JOIN orders o ON oi.order_id = o.order_id JOIN products p ON oi.product_id = p.product_id ORDER BY oi.order_id;`

order_id	product_name	quantity	unit_price	discount_pct
1001	Wireless Earbuds	2	1499.00	0.00
1001	Laptop Stand	1	899.00	10.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1004	Bluetooth Speaker	1	3499.00	0.00
1005	Smart Watch	1	2999.00	0.00
1006	Wireless Earbuds	1	1499.00	10.00
1006	Running Shoes	1	4599.00	5.00
1007	Bedsheet Set	1	1299.00	0.00
1008	Laptop Stand	1	899.00	0.00
1009	Bluetooth Speaker	1	3499.00	0.00
1009	Cushion Covers (...)	2	599.00	15.00
1010	Bedsheet Set	1	1299.00	0.00
1010	Cushion Covers (...)	1	599.00	0.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

Ans.- **LEFT JOIN** returns all rows from the LEFT table and matching rows from the right. If there is no match, right-side columns are NULL.

Example from this schema: **SELECT c.first_name, o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;**

first_name	order_id
Aarav	1001
Aarav	1004
Priya	1002
Rohan	1003
Rohan	1008
Sneha	1005
Vikram	1006
Ananya	1007
Karan	1009
Divya	1010

RIGHT JOIN is the mirror: all rows from the RIGHT table, with NULLs for unmatched left-side columns. Most databases allow rewriting a RIGHT JOIN as a LEFT JOIN by swapping the table order.

Example from this schema: **SELECT c.first_name, o.order_id FROM customers c RIGHT JOIN orders o ON c.customer_id = o.customer_id;**

first_name	order_id
Aarav	1001
Aarav	1004
Priya	1002
Rohan	1003
Rohan	1008
Sneha	1005
Vikram	1006
Ananya	1007
Karan	1009
Divya	1010

FULL OUTER JOIN returns all rows from BOTH tables, with NULLs wherever no match exists. Use it when you need to find unmatched rows on either side, for example, detecting both customers with no orders AND orders with no matching customer (orphaned records).

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

Ans.- The three Foreign Key relationships in the schema:

- orders.customer_id → customers.customer_id
- order_items.order_id → orders.order_id
- order_items.product_id → products.product_id

if we try to insert an order with customer_id = 999 using- **INSERT INTO orders VALUES (1011, 999, '2024-08-31', 'Pending', 500.00);**

Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('shopease_db`.`orders`, CONSTRAINT `orders_ibfk_1` FOREIGN KEY ('customer\_id') REFERENCES `customers` ('customer_id'))

The database enforces referential integrity: customer_id 999 does not exist in the customers table, so the INSERT is rejected. This prevents orphaned records and ensures every order always belongs to a valid customer.

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

Ans.- `SELECT product_name, unit_price, CASE WHEN unit_price < 1000 THEN 'Budget' WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range' ELSE 'Premium' END AS price_tier FROM products;`

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

Ans.- `SELECT SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS delivered_orders, SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS not_delivered_orders FROM orders;`

delivered_orders	not_delivered_orders
6	4

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

Ans.- ACID is a set of four properties that guarantee reliable database transactions:

A — Atomicity

All operations in a transaction are treated as a single unit. Either ALL succeed or ALL are rolled back. Example: In a bank transfer, debiting Account A and crediting Account B must both happen, if the credit fails, the debit is also undone. No money is lost in the middle.

C — Consistency

A transaction brings the database from one valid state to another. All defined rules (constraints, foreign keys, check conditions) must hold before and after. Example: A bank transfer must not result in a negative balance if the CHECK constraint prohibits it.

I — Isolation

Concurrent transactions execute as if they were serial, they do not see each other's intermediate states.

Example: If two people simultaneously try to book the last seat on a flight, isolation ensures only one succeeds; the other sees the updated seat count.

D — Durability

Once a transaction is committed, its changes are permanent even if the system crashes immediately afterward.

The database uses write-ahead logs (WAL) and disk flushes to guarantee this.

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products
4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

Write the complete BEGIN...COMMIT/ROLLBACK block.

Ans.- **BEGIN;**

-- Step 1: Insert new order

```
INSERT INTO orders (order_id, customer_id, order_date, status, total_amount)
VALUES (1011, 102, CURRENT_DATE, 'Pending', 1598.00);
```

order_id	customer_id	order_date	status	total_amount
1011	102	2026-05-31	Pending	1598.00
NULL	NULL	NULL	NULL	NULL

-- Step 2a: First order item (Bedsheet Set, qty 1)

```
INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price, discount_pct)
VALUES (5016, 1011, 206, 1, 1299.00, 0);
```

item_id	order_id	product_id	quantity	unit_price	discount_pct
5016	1011	206	1	1299.00	0.00
NULL	NULL	NULL	NULL	NULL	NULL

-- Step 2b: Second order item (Cushion Covers, qty 1)

```
INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price, discount_pct)
VALUES (5017, 1011, 208, 1, 599.00, 0);
```

item_id	order_id	product_id	quantity	unit_price	discount_pct
5016	1011	206	1	1299.00	0.00
5017	1011	208	1	599.00	0.00
NULL	NULL	NULL	NULL	NULL	NULL

-- Step 3a: Reduce stock for Bedsheet Set

UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206;

product_id	product_name	stock_qty
206	Bedsheet Set	299
208	Cushion Covers (Set)	400
NULL	NULL	NULL

-- Step 3b: Reduce stock for Cushion Covers

UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208;

product_id	product_name	stock_qty
206	Bedsheet Set	299
208	Cushion Covers (Set)	399
NULL	NULL	NULL

COMMIT;

-- If any step fails: **ROLLBACK;** (undoes all changes atomically)

This transaction is atomic: if any of the 5 statements fails (e.g., invalid product_id, stock goes negative), issuing ROLLBACK reverts every change made so far. If all succeed, COMMIT makes them permanent.

Brief Business Insights — ShopEase E-Commerce (August 2024)

Based on the SQL analysis of ShopEase's August 2024 data:

1. Revenue Overview

- Total revenue from all orders: **₹35,085**
- Revenue from Delivered orders only: **₹17,191** (49% of total)
- ₹13,596 is still in **Shipped** status — potential future revenue
- ₹2,999 lost to **1 Cancelled** order (Sneha Reddy's Smart Watch)

2. Category Performance

- Clothing has the highest average price: **₹2,699** (driven by Running Shoes at ₹4,599)
- Electronics average: **₹2,224**
- Home is the budget category: **₹949 average**
- Only Clothing and Electronics exceed the ₹2,000 average threshold

3. Top Customers

- **Rohan Gupta** placed the highest single order: ₹7,498
- **Aarav Sharma** is the most active: 2 orders totalling ₹7,997
- Both are **premium members** — suggests premium customers drive more revenue

4. Product Insights

- Most ordered products: Running Shoes and Bluetooth Speaker (appear in multiple orders)
- Budget products (< ₹1,000): Cotton T-Shirt, Laptop Stand, Cushion Covers
- Premium products (> ₹3,000): Running Shoes (₹4,599), Bluetooth Speaker (₹3,499)

5. Order Status Distribution

Status	Orders	Revenue
Delivered	6	₹17,191
Shipped	2	₹13,596
Cancelled	1	₹2,999
Pending	1	₹1,299

60% of orders are delivered, healthy fulfillment rate for one month.

6. Data Quality Validation

- Total customers: **8** — no duplicates (UNIQUE email enforced)
- Total products: **8** — all have valid positive prices (CHECK constraint)
- Total orders: **10** — all linked to valid customers (FK enforced)
- Total order items: **15** — all linked to valid orders and products
- No NULL values found in any NOT NULL column

7. Key Recommendations

- Follow up on the **1 Cancelled order** (₹2,999) — understand why the Smart Watch was cancelled
- The **2 Shipped orders** (₹13,596) should convert to Delivered — track delivery status
- **Premium members** (Aarav, Rohan, Vikram, Karan) order more — consider loyalty rewards
- **Home category** has low average price but high stock — consider bundling or promotions